

TECHNICAL OVERVIEW DOCUMENT

Enterprise Agentic RAG

Knowledge Assistant

Intelligent Enterprise Knowledge Retrieval System

Project Title	Intelligent Enterprise Knowledge Retrieval System
Project Type	Enterprise Agentic RAG System
Prepared By	Thusharkanth Loganathan
Role	AI/ML Engineer Intern
Date	May 2026
Project Status	Prototype / Working MVP
Repository	GIT REPO LINK
Version	Version 1.0

1. Executive Summary

Modern enterprises generate large volumes of internal knowledge across HR policies, onboarding manuals, cybersecurity procedures, deployment guides, AI governance policies, and operational SOPs. Traditional Large Language Models cannot directly access this private organizational knowledge and frequently produce hallucinated or inaccurate responses.

This document presents the Enterprise Agentic RAG Knowledge Assistant — an intelligent system that retrieves relevant enterprise knowledge from internal documents and generates grounded, source-cited responses. The system is built on an agentic architecture orchestrated by LangGraph, combining semantic retrieval, LLM-based evaluation, and conditional answer generation.

Core Technologies

- LangGraph — agentic workflow orchestration and conditional routing
- ChromaDB — local persistent vector store for semantic search
- Ollama (nomic-embed-text) — local embedding model for vector generation
- Google Gemini 1.5 Flash — LLM for evaluation and response generation
- Streamlit — lightweight frontend user interface

Key Capabilities

- Retrieval-Augmented Generation over private enterprise documents
- LLM-based context evaluation replacing heuristic grading
- Conditional routing — grounded answer or graceful fallback
- Source citations appended to every response
- Hallucination prevention through retrieval grounding

2. Problem Statement

Organisations face a persistent challenge in making internal knowledge accessible. Documentation is scattered across multiple systems, retrieval is manual and time-consuming, and deploying standard LLMs introduces risks of inaccurate, ungrounded responses.

Challenges This System Addresses
– LLMs with no access to internal documents produce hallucinated answers
– Compliance and audit risks from ungrounded AI-generated responses
– High support overhead from repetitive HR, IT, and onboarding queries
– Scattered documentation requiring manual effort to locate answers
– Lack of explainability and traceability in standard LLM responses

The goal is an intelligent knowledge assistant that retrieves verified enterprise information dynamically, grounds every response in internal documents, and makes its reasoning transparent through source citations.

3. Project Objectives

1. Build an enterprise-grade RAG pipeline with a full document ingestion workflow
2. Implement LangGraph-based agentic workflow orchestration
3. Integrate semantic document retrieval using ChromaDB vector store
4. Use Ollama embeddings for local, privacy-preserving vector generation
5. Implement LLM-based context evaluation using Google Gemini
6. Generate grounded answers with source document citations
7. Prevent hallucinations through retrieval augmentation
8. Demonstrate agentic AI workflow concepts applicable to enterprise environments

4. Business Value

The system delivers measurable value across multiple enterprise departments by enabling instant, grounded, and auditable access to internal knowledge.

Key Benefits
– Improved Knowledge Accessibility — Employees instantly access enterprise information without manually searching documents
– Reduced Support Overhead — Fewer repetitive HR, onboarding, compliance, and IT support queries
– Improved Trustworthiness — Answers grounded in internal company documents with source citations
– Faster Employee Onboarding — New employees quickly retrieve onboarding and policy information
– AI Governance and Compliance — Transparent retrieval-based responses support enterprise AI governance

Potential Enterprise Use Cases

- HR Policy and Benefits Assistant
- Employee Onboarding Knowledge Base
- Cybersecurity SOP and Incident Response Assistant
- Internal Compliance and Audit Assistant
- AI Governance Knowledge Assistant
- Deployment Procedure Reference Tool

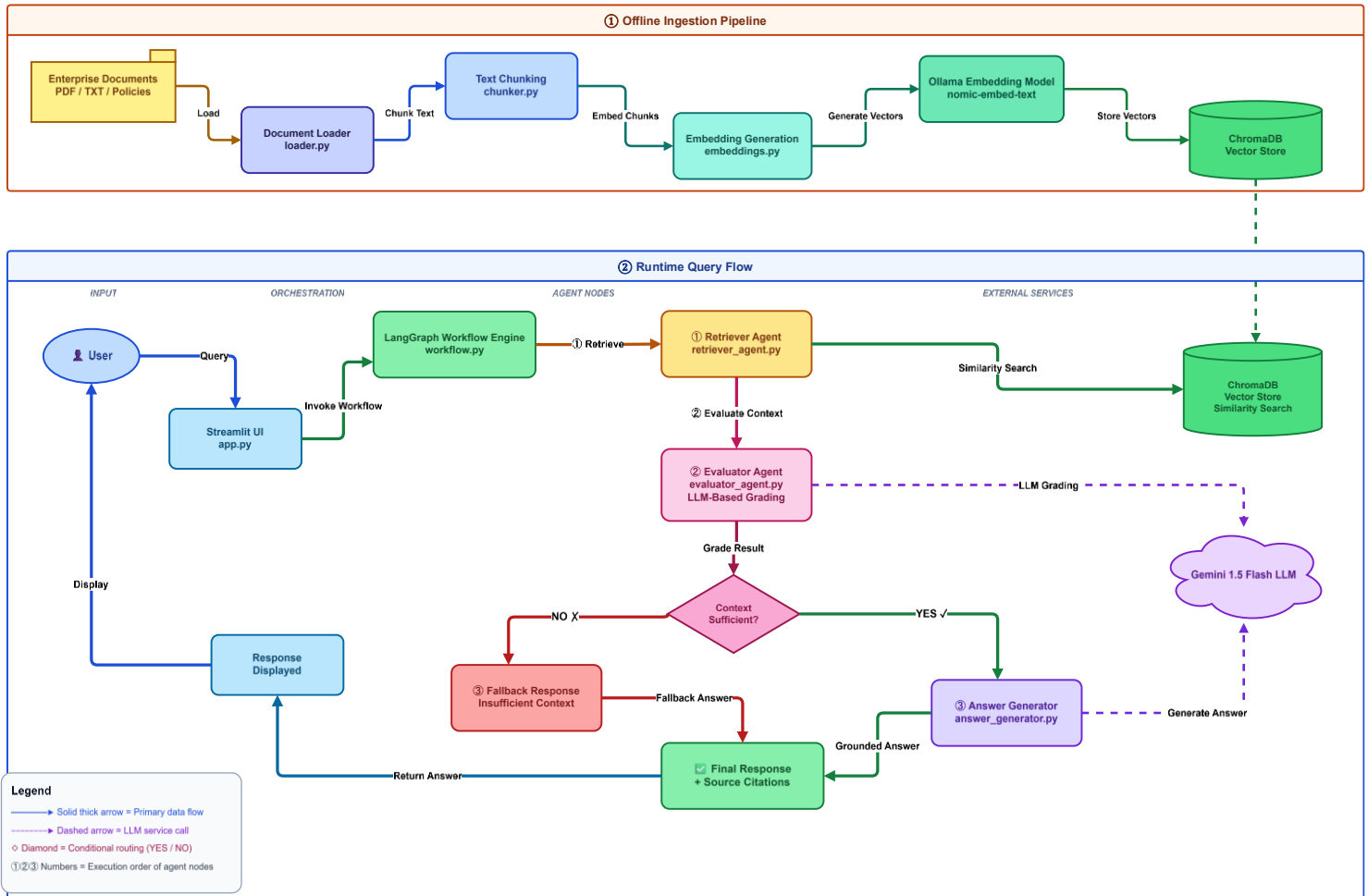
5. Technology Stack

Technology	Version / Model	Purpose
Python	3.11+	Core programming language
Streamlit	Latest	Frontend web UI framework
LangGraph	Latest	Agentic workflow orchestration and state management
LangChain	Latest	LLM abstraction, document loading, and retrieval utilities
Google Gemini	Gemini 1.5 Flash	Primary LLM for evaluation and answer generation
Ollama	nomic-embed-text:latest	Local embedding model for vector generation
ChromaDB	Latest	Local persistent vector store for semantic search

6. System Architecture

The system follows a modular, layered enterprise AI architecture. Each layer has a clearly defined responsibility and communicates through well-defined interfaces, enabling independent testing and future extensibility.

Enterprise Agentic RAG — System Architecture



6.1 UI Layer

Implemented using Streamlit. Responsible for user interaction, query input, and rendering generated answers with source citations. Provides a clean, browser-accessible interface requiring no installation.

6.2 Orchestration Layer

Implemented using LangGraph. Responsible for defining the agent graph, managing typed state, executing nodes in sequence, and applying conditional routing based on evaluation results. All agent transitions are explicit and auditable.

6.3 Agent Layer

Agent	File	Responsibility
Retriever Agent	retriever_agent.py	Performs semantic similarity search against ChromaDB to fetch the most relevant document chunks
Evaluator Agent	evaluator_agent.py	Uses Gemini LLM to determine whether retrieved context is sufficient to answer the query
Answer Generator	answer_generator.py	Generates a grounded, source-cited response from retrieved enterprise context using Gemini
Fallback Handler	nodes.py	Returns a graceful fallback message when the evaluator determines context is insufficient

6.4 Retrieval Layer

Uses ChromaDB for persistent local vector storage. At query time, the user's question is embedded using Ollama and compared against stored document vectors via cosine similarity. The top-k most relevant chunks are returned to the agent pipeline.

6.5 AI Layer

Uses Google Gemini 1.5 Flash for two distinct tasks: context evaluation (determining sufficiency) and answer generation (producing grounded responses). Ollama handles embedding generation locally for both ingestion and runtime retrieval.

6.6 Data Layer

Contains 16 synthetic enterprise documents covering HR policies, onboarding guides, cybersecurity SOPs, deployment workflows, AI governance policies, data governance, vendor management, and performance management. Documents intentionally contain partial overlaps and incomplete information to challenge retrieval quality and agent reasoning.

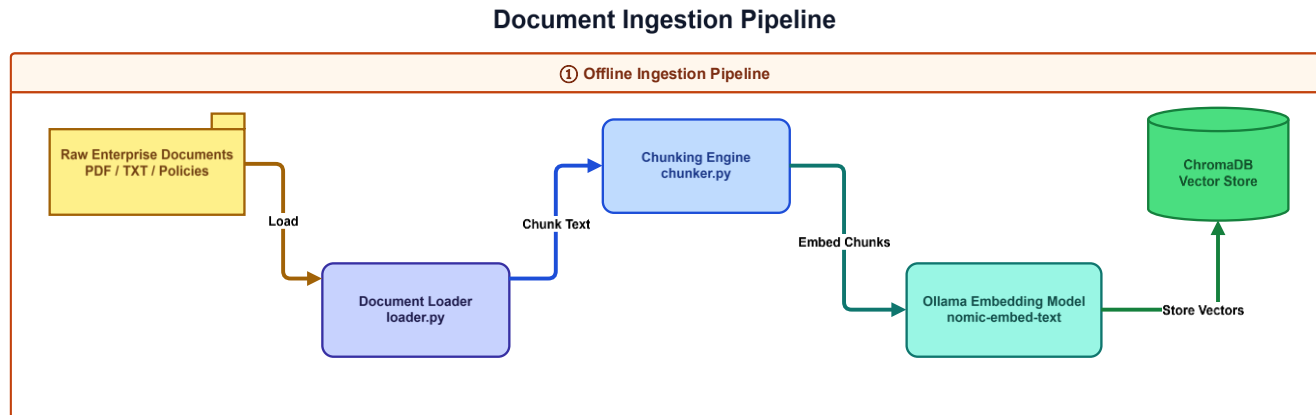
7. Repository Structure

```
enterprise-agentic-rag/
├── app.py                # Streamlit frontend entry point
├── requirements.txt      # Python dependencies
├── .env                 # Environment variables (API keys)
├── data/
│   ├── raw/             # Raw enterprise documents (.txt)
│   └── processed/       # Post-ingestion artefacts
├── ingestion/
│   ├── ingest.py        # Ingestion pipeline entry point
│   ├── loader.py        # Document loading (DirectoryLoader)
│   ├── chunker.py       # Text chunking (RecursiveCharacterTextSplitter)
│   └── embedder.py      # Embedding generation (Ollama)
├── vectorstore/
│   ├── chroma_db/       # Persisted ChromaDB vector store
│   └── vectordb.py      # Vector store interface
├── agents/
│   ├── retriever_agent.py # Semantic retrieval agent
│   ├── evaluator_agent.py # LLM-based context grading agent
│   ├── answer_generator.py # Grounded response generation
│   ├── query_analyzer.py  # [Placeholder - future]
│   └── web_search_agent.py # [Placeholder - future]
├── graph/
│   ├── workflow.py      # LangGraph graph definition
│   ├── state.py         # Typed state schema
│   └── nodes.py         # Node implementations
├── services/
│   ├── llm.py           # Gemini LLM configuration
│   ├── embeddings.py    # Ollama embeddings service
│   └── retrieval.py      # Retrieval service wrapper
├── prompts/            # Prompt templates
├── diagrams/           # Architecture diagrams
└── docs/               # Supporting documentation
```

GIT repo Link - <https://github.com/Thusharkanth/enterprise-agentic-rag>

8. Document Ingestion Pipeline

The ingestion pipeline is an offline process run once prior to deployment. It converts raw enterprise documents into semantic vector embeddings and persists them in ChromaDB for runtime retrieval.



Step 1 — Document Loading

Documents are loaded from data/raw/ using LangChain's DirectoryLoader and TextLoader, recursively scanning for .txt files across all enterprise document categories.

Step 2 — Text Chunking

Documents are split using RecursiveCharacterTextSplitter with the following parameters:

```
CHUNK_SIZE      = 500      # characters per chunk
CHUNK_OVERLAP   = 100      # character overlap between adjacent chunks
```

Chunking improves retrieval granularity by ensuring each retrieved piece of text is focused and semantically dense, rather than retrieving entire documents.

Step 3 — Embedding Generation

Each chunk is passed through Ollama's nomic-embed-text model to produce a dense, 768-dimensional semantic vector. Embeddings are generated locally, ensuring data privacy.

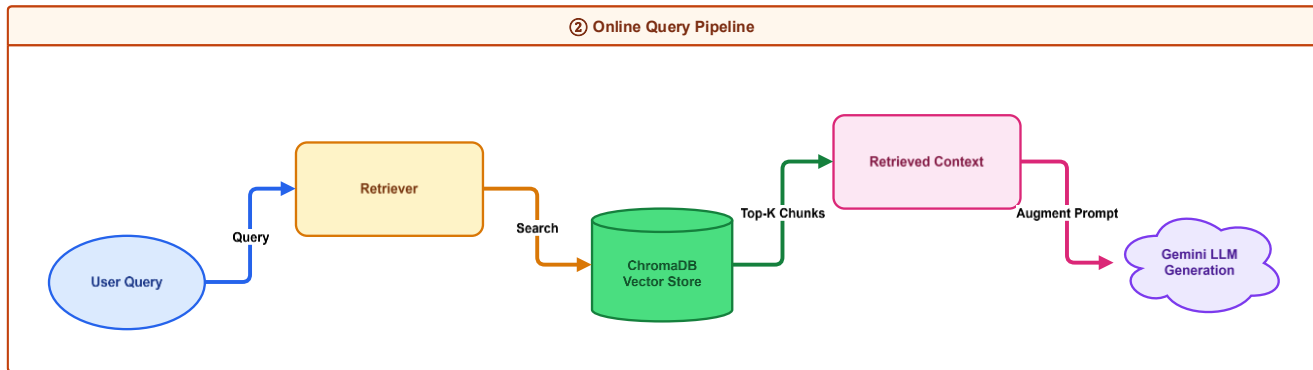
Step 4 — Vector Storage

Generated embeddings are stored in ChromaDB, persisted at vectorstore/chroma_db/. ChromaDB provides efficient approximate nearest-neighbour search for fast runtime retrieval.

9. RAG Pipeline

The Retrieval-Augmented Generation pipeline grounds all LLM responses in enterprise document content, eliminating hallucinations and ensuring answers are traceable to specific source documents.

RAG Pipeline Flow



Step 1 — User Query

The user submits a natural language question through the Streamlit UI. Example:

```
How should employees report security incidents?
```

Step 2 — Retrieval

The Retriever Agent embeds the query using Ollama and performs cosine similarity search against ChromaDB. The top-k most semantically relevant document chunks are returned.

Step 3 — Augmentation

Retrieved document chunks are injected into the LLM prompt as context. This grounds the model in verified enterprise knowledge rather than relying on pretrained weights.

Step 4 — Evaluation

The Evaluator Agent uses Gemini to assess whether the retrieved context is sufficient to accurately answer the query. It returns either sufficient or insufficient.

Step 5 — Generation or Fallback

If context is sufficient, the Answer Generator produces a grounded response with source citations appended. If insufficient, a graceful fallback message is returned, preventing hallucination.

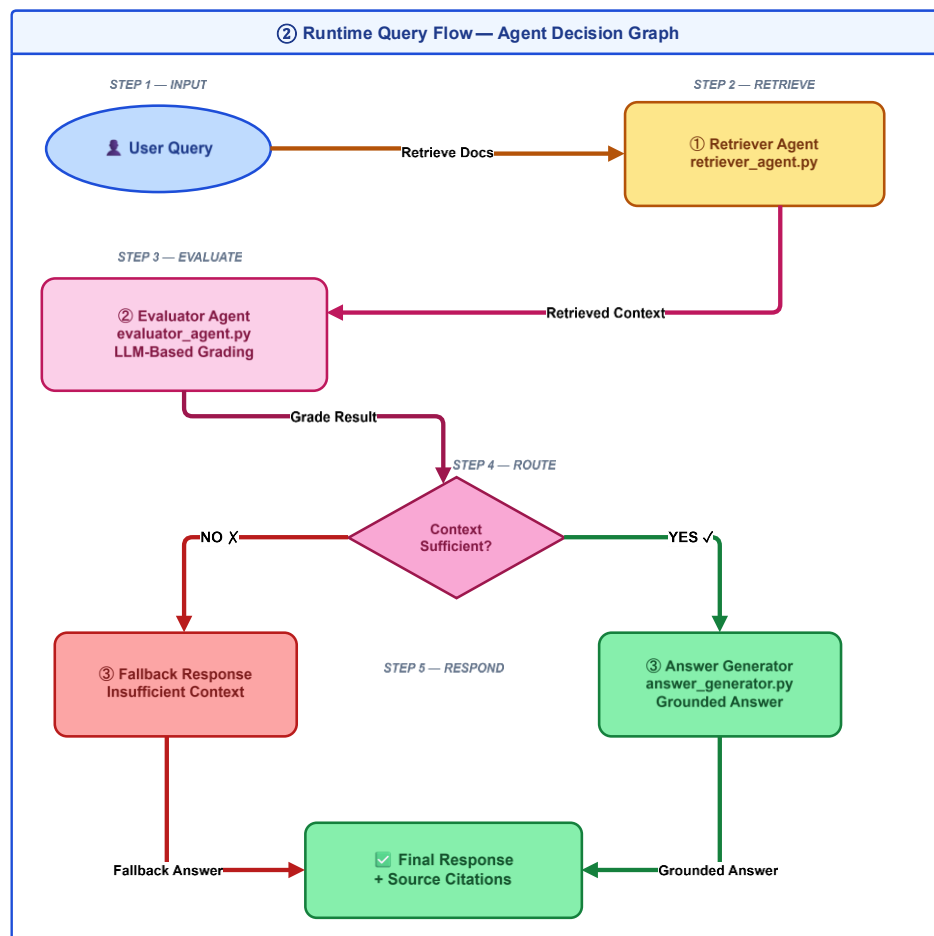
Advantages of RAG over Standard LLMs

- Eliminates hallucinations by anchoring responses in retrieved documents
- Enables access to private enterprise knowledge without external API exposure
- Supports dynamic knowledge updates by re-ingesting new documents
- Avoids costly fine-tuning for domain-specific knowledge adaptation
- Delivers explainability through automatic source citations

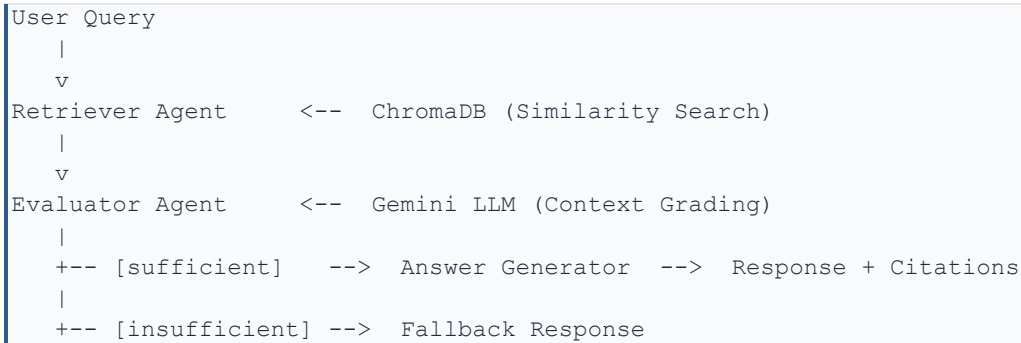
10. Agentic Workflow

The system implements a true agentic workflow using LangGraph. Each processing step is a distinct graph node with typed inputs and outputs. The workflow applies conditional routing based on LLM evaluation results — a key distinction from static RAG pipelines.

Agentic Decision Workflow



10.1 Workflow Execution Sequence



10.2 Retriever Agent

Responsible for semantic retrieval of the most relevant document chunks from ChromaDB. Embeds the user query using Ollama and retrieves the top-k chunks by cosine similarity score.

10.3 Evaluator Agent

Uses Gemini to intelligently assess whether the retrieved context contains sufficient information to answer the query. Replaces simple keyword heuristics with genuine LLM reasoning, returning sufficient or insufficient as a structured signal for the router.

10.4 Conditional Routing

LangGraph applies conditional routing based on the evaluator's output. This creates dynamic, decision-driven workflow execution — the defining characteristic of an agentic system as opposed to a fixed sequential pipeline.

10.5 Answer Generator

Generates a grounded, accurate answer using Gemini with retrieved document chunks as context. Automatically formats and appends source document filenames as citations.

11. Source Citation Strategy

Every generated answer includes the source documents from which the response was constructed. This provides full traceability and supports enterprise audit and compliance requirements.

EXAMPLE OUTPUT

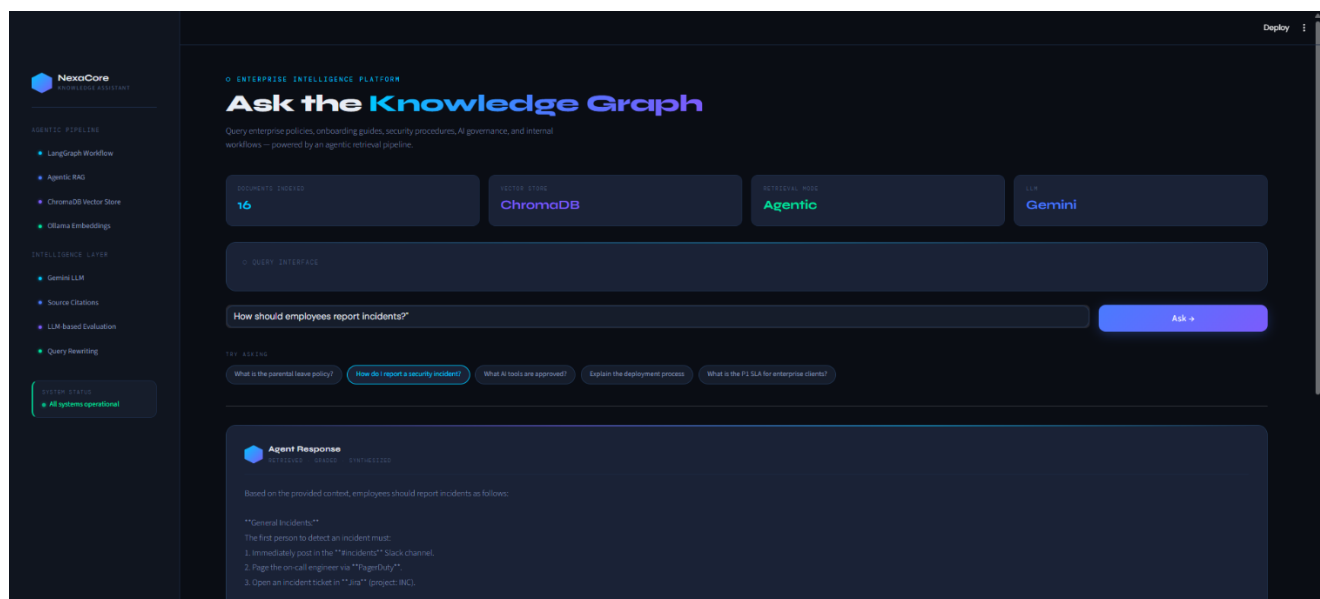
```
Sources:
- 09_incident_management_workflow.txt
- 06_cybersecurity_guidelines.txt
```

Citation Benefits

- Explainability — Users know exactly which documents informed the answer
- Transparency — Every claim is traceable to a specific source document
- Compliance — Supports audit trails and enterprise governance requirements
- Trust — Reduces user scepticism by exposing the evidence base

12. Testing & Validation

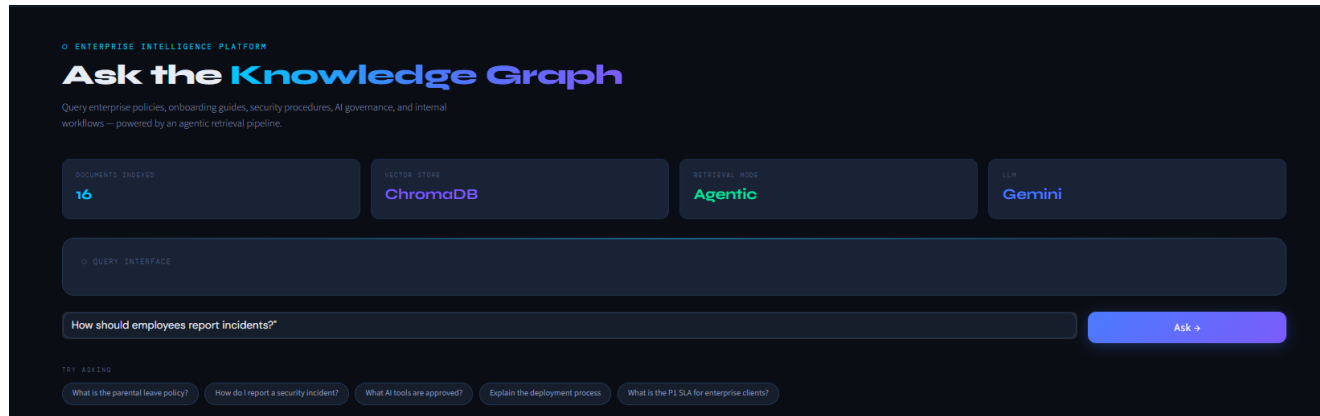
The system was validated across a range of queries covering successful retrieval scenarios and deliberate failure cases. Both paths of the conditional router were tested and confirmed functional.



Successful Retrieval

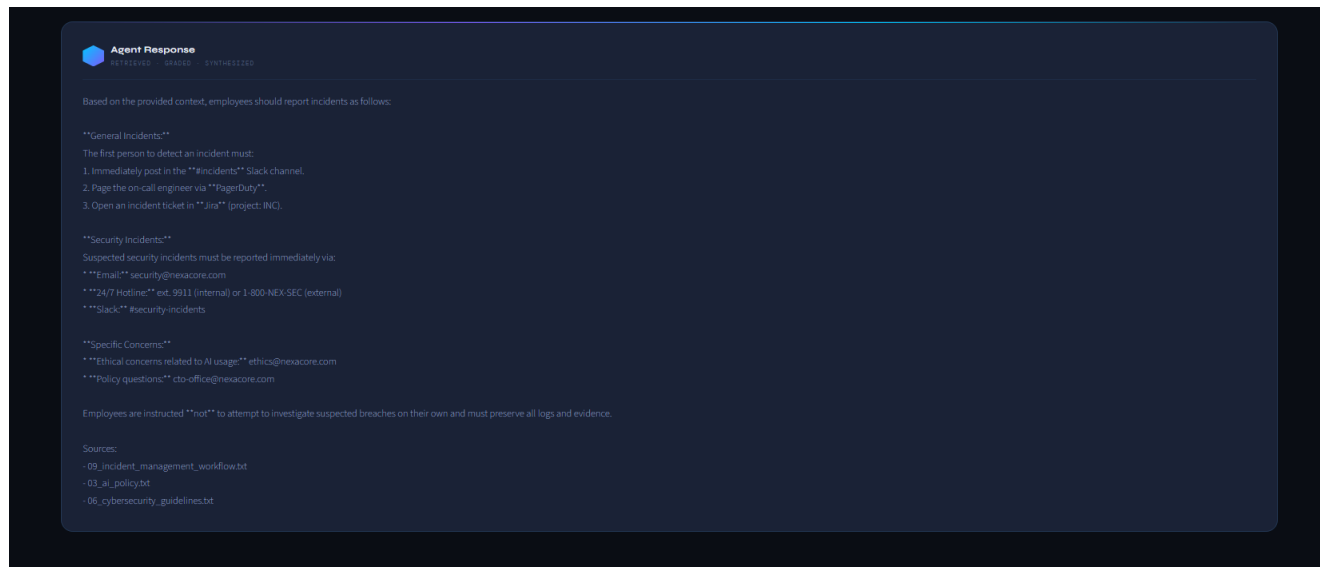
Query

How should employees report security incidents?



Outcome

- Retriever fetched relevant chunks from incident management and cybersecurity documents
- Evaluator graded context as sufficient
- Answer Generator produced a grounded, step-by-step response
- Citations appended: 09_incident_management_workflow.txt, 06_cybersecurity_guidelines.txt



Hallucination Prevention Case

Query

```
|What is the company revenue for 2027?
```

Outcome

```
|I could not find sufficient information in the enterprise knowledge base.
```

The evaluator correctly identified that no relevant context existed in the vector store. The conditional router triggered the fallback handler, returning a graceful message and preventing a hallucinated response.

13. Current Limitations

The following limitations are acknowledged. Several components are intentional placeholders scoped for future development cycles.

Limitation	Description
Deterministic Workflow	Fixed retrieve → evaluate → generate path. Multi-agent collaboration and parallel execution not yet implemented.
Duplicate Retrieval	The Answer Generator re-fetches documents rather than reusing context already held in the LangGraph state, introducing minor inefficiency.
Placeholder Modules	<code>query_analyzer.py</code> , <code>web_search_agent.py</code> , and several prompt templates are stubs intended for future implementation.
No Authentication	The system does not implement user authentication or role-based access control (RBAC) in its current form.
Local Dependency	Requires a locally running Ollama instance and a downloaded embedding model, limiting cloud-native deployment options.

14. Future Improvements

The following enhancements would advance this system towards a production-grade enterprise platform:

- Hybrid search combining BM25 keyword retrieval with dense vector search for improved recall
- Web search integration for queries that fall outside the enterprise document corpus
- Multi-agent collaboration with specialist agents for HR, security, compliance, and IT domains
- Conversational memory enabling multi-turn dialogue with session context retention
- User authentication and role-based access control governing document-level permissions
- Observability and logging providing full tracing of agent decisions and retrieval quality
- Automated evaluation metrics using the RAGAS framework for continuous quality assessment
- Cloud deployment via containerised workloads on AWS, Azure, or GCP

- Real-time document indexing with automatic re-ingestion triggered by document updates

15. Conclusion

This project demonstrates the design and implementation of an enterprise-grade Agentic RAG knowledge assistant combining LangGraph workflow orchestration, ChromaDB semantic retrieval, Ollama local embeddings, and Google Gemini reasoning.

The system successfully integrates vector search, agentic decision-making, LLM-based context evaluation, grounded generation, and source citation into a cohesive, modular architecture. It demonstrates core enterprise AI engineering concepts — Retrieval-Augmented Generation, agent orchestration, semantic retrieval, explainable AI, and conditional workflow execution — providing a strong foundation for scalable enterprise AI assistants and future multi-agent knowledge systems.

Project Highlights

- 16 synthetic enterprise documents with intentional overlaps and knowledge gaps
- LangGraph agentic workflow with intelligent LLM-based conditional routing
- Local Ollama embeddings preserving data privacy during vector generation
- Source-cited, grounded responses preventing hallucination at every query
- Fully modular architecture designed for future extension and cloud deployment